

Documento tecnico come è costruito QUANTO

Architettura, processi di ogni funzione, e come è fatto il codice — per chi si unisce al progetto.

26 settembre 2026 · <https://quanto-self.vercel.app> · repository GitHub, ramo `main`

Questo documento affianca, non sostituisce, `README.md` e `docs/DEPLOY.md` nel repository: lì ci sono i comandi esatti per avviare il progetto e le variabili d'ambiente da impostare. Qui c'è il perché delle scelte e la mappa di come tutto si tiene insieme.

Indice

1. Cos'è QUANTO e da cosa nasce

2. Architettura generale

3. Il flusso end-to-end

4. Scaletta dei processi, funzione per funzione

5. Come sono state costruite le parti già fatte
6. Come tutto è collegato

7. Come è composta la pagina (frontend)

8. Stato attuale, in una tabella

9. Convenzioni per chi ci lavora

10. Da dove continuare

1. Cos'è QUANTO e da cosa nasce

QUANTO (prima chiamato FKOS, *Financial Knowledge Operating System*) è un motore che controlla se il budget di un progetto rispetta le regole di un bando pubblico (europeo, nazionale o regionale), lo confronta con i budget di progetti già finanziati, distribuisce la spesa tra più fondi compatibili e certifica il risultato in modo che non si possa alterare in seguito senza che si veda.

Il progetto nasce da due documenti che restano il riferimento concettuale (in `docs/` e nello scratchpad di lavorazione): la **specificazione FKOS v2** — che descrive 13 moduli funzionali, i 60 criteri di controllo e il funzionamento dell'asseverazione — e la **Mappa dei processi**, uno schema più operativo che elenca, modulo per modulo, cosa dovrebbe succedere passo passo. Quest'ultima è servita anche come lista di cose da sistemare (la sua "Parte D") nella fase più recente del progetto: passaggio a un database vero, dati ufficiali reali al posto di tabelle di esempio, autenticazione vera, integrazione dell'IA dove la specifica la prevede.

Da questi due documenti restano tre principi che **non si negoziano**, applicati in ogni riga di codice:

1. **Niente dati finti.** Se un valore non è stato dichiarato da una fonte ufficiale (un bando pubblicato, una tabella CCNL attestata, un documento caricato dal cliente), il criterio che ne avrebbe bisogno resta *"non valutato"* — non si inventa un default plausibile, e non si finge un calcolo. Questo vale per i 60 criteri, per Fonte B, per la banca dei pattern: se manca il dato, il sistema lo dice, non lo aggira.

2. **Denaro sempre in `Decimal`, mai in `float`.** Un errore di arrotondamento binario su un importo di progetto è un errore reale, non un dettaglio: tutto il motore usa `decimal.Decimal` di Python; il frontend non fa mai un calcolo monetario, mostra solo quello che il server ha calcolato.

3. **Niente blockchain** (richiesta esplicita del committente originale). Al suo posto, un registro a catena di hash firmato (Ed25519), append-only anche a livello di database: stessa garanzia di immutabilità verificabile, senza le complicazioni di un registro distribuito che qui non serve.

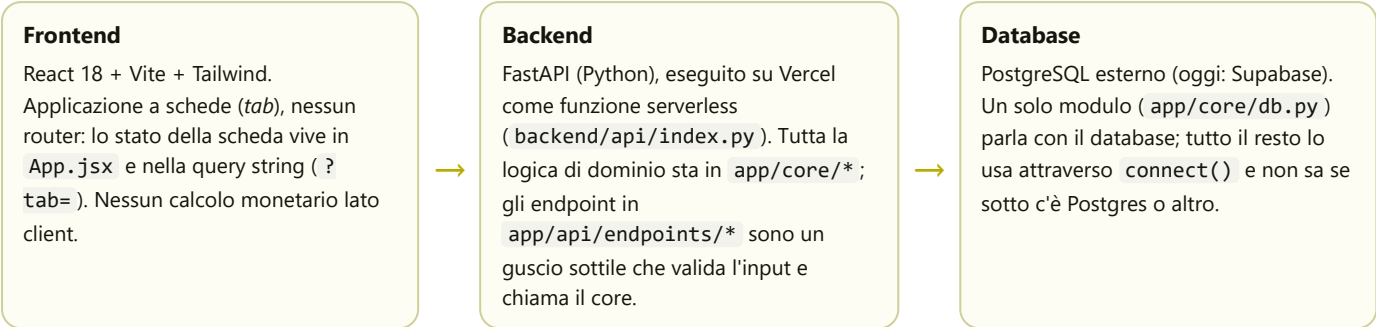
Il motore serve due "missioni" distinte, che corrispondono a due pagine diverse dell'app:

Missione	Cosa fa	Pagina / endpoint
Missione Uno	Controlla un budget riga per riga contro le regole di <u>un</u> bando: 60 criteri deterministici, ognuno o "superato/non superato" o "non valutato" se manca il dato.	pagina Budget · <code>POST /api/v2/budget/validate</code>
Missione Due	Dato un budget già validato (o un bilancio storico), distribuisce ogni spesa tra i fondi compatibili disponibili, rispettando tetti, quote, non cumulabilità e finestre temporali (MILP).	pagina Allocazione · <code>POST /api/v2/allocation/optimize</code>

Attorno a queste due, tre funzioni di contorno altrettanto centrali: la **libreria dei bandi** (da dove arrivano le regole, Fonte A), il **confronto con i budget premiati** (banca dei pattern, k-means) e l'**asseverazione** (Merkle + registro firmato, per provare che un budget certificato non è stato toccato dopo).

2. Architettura generale

Tre pezzi, distribuiti su un solo progetto Vercel:



Due servizi esterni opzionali, entrambi spenti finché non si configura una chiave — il sistema funziona (in modo più manuale) anche senza:

Anthropic (Claude)
Solo per: arricchire i metadati di un bando scoperto, leggere un documento di bando con più passaggi indipendenti, scrivere un riepilogo in prosa di un risultato già calcolato. Mai per calcolare un numero: quello lo fa sempre il motore deterministico.

Motore OCR
Serve solo per le scansioni (immagini) nei documenti del cliente; il testo dei PDF normali si legge senza. L'interfaccia è pronta (`app/core/fonte_c/ocr.py`), Tesseract è collegato ma non gira su Vercel: va scelto un servizio cloud.

Distribuzione (Vercel)

Percorso	Come è servito
<code>/api/v2/*</code> , <code>/docs</code> , <code>/openapi.json</code>	Funzione Python (<code>backend/api/index.py</code> , che monta la app FastAPI di <code>backend/main.py</code>)
<code>/assets/*</code> , tutto ciò con un'estensione	File statici della build Vite (<code>frontend/dist</code>)
qualunque altro percorso	<code>frontend/index.html</code> (SPA: la scheda la sceglie il codice React, non il server)
Cron giornaliero (03:00)	GET <code>/api/v2/cron/catalog-refresh</code> , protetto da <code>CRON_SECRET</code>

Configurazione: `vercel.json` (root del repository) · variabili d'ambiente e passi di messa in produzione: `docs/DEPLOY.md`.

Il percorso di una richiesta, dall'inizio alla fine

Esempio concreto — l'utente preme "Controlla" nella pagina Budget:

- `BudgetCanvas.jsx` chiama `api.validateBudget(request)` (in `frontend/src/lib/api.js`)
- `api.js` fa una `fetch` verso `/api/v2/budget/validate` , con il token della sessione nell'header `Authorization`
- `app/api/router.py` instrada alla dipendenza `require_auth` (`app/api/deps.py`), poi all'endpoint in `app/api/endpoints/budget.py`
- l'endpoint valida la forma della richiesta (Pydantic, `app/models/schemas.py`) e chiama `app/core/analysis.py`
- `analysis.py` apre lo *scope* di Fonte B per la data di riferimento, poi per ogni voce chiama `deterministic_engine.py` (i 60 criteri) e infine `merkle_tree.py` per la radice
- se è configurata una chiave del modello, uno stadio in più (`llm.py` , con verifica delle citazioni) arricchisce il riepilogo testuale
- ogni passaggio scrive un evento (`events.record`) e la traccia completa in `runs` , per la ripresa dal Quartier Generale
- la risposta torna al frontend, che aggiorna lo stato React e mostra il risultato — nessun numero è ricalcolato lì

3. Il flusso end-to-end

Nel linguaggio della specifica originale, i dati arrivano da tre "Fonti" e alimentano le due "Missioni"; il risultato viene certificato e, per la Missione Uno, confrontato con lo storico. Questa è la catena completa, nell'ordine in cui un progetto la attraversa davvero:

Fase	Cosa fa	Moduli principali	Stato
Fonte A	Trova il bando: ricerca interna, conferma, cache; se serve, ricerca sul web e lettura dei documenti ufficiali	bandi.py · discovery.py · research.py · requirements_extractor.py	Fatto
Fonte B	Tabelle ufficiali versionate (CCNL, oneri, ammortamenti, benchmark), bozza → attestazione → pubblicazione	fonte_b.py · fonte_b_admin.py	Fatto, tabelle non ancora caricate
Fonte C	Documenti del cliente (buste paga, bilanci, F24): cifratura, lettura, confidenza per campo, revisione	fonte_c/ocr.py · parsers.py · service.py	Fatto, OCR da collegare
Missione Uno	Valida il budget contro i 60 criteri del bando scelto	deterministic_engine.py · criteria_catalog.py · analysis.py	Fatto
Asseverazione	Impronta Merkle del budget, registrazione firmata, verifica indipendente	merkle_tree.py · registry.py	Fatto
Confronto	Scostamento della ripartizione di spesa rispetto agli archetipi di budget premiati	pattern_bank.py	Fatto, banca dati vuota
Missione Due	Distribuisce la spesa tra i fondi compatibili (MILP)	allocation_engine.py · funds.py	Fatto
Quartier Generale	Supervisione: eventi, run, archivio bandi, utenti, database	hq.py · events.py · archive.py · users.py	Fatto

4. Scaletta dei processi, funzione per funzione

Per ogni funzione: a cosa serve, i passi esatti del processo, dove sta il codice, e a che punto è.

4.1 — Autenticazione e utenti Fatto

1. Al primo avvio, se non esiste nessun utente, `bootstrap_admin()` crea l'unico manager iniziale da due variabili d'ambiente, poi si disattiva da sola.
2. Login: verifica script della password; dopo 5 tentativi falliti l'account si blocca 15 minuti (contatore nel database, quindi vale anche con più istanze del server).
3. Token: claims firmati con `token_version`; cambiare la password o disattivare l'utente fa scadere all'istante ogni token già emesso, senza bisogno di una lista nera.
4. Ogni richiesta protetta passa da `require_auth`: prima il token utente, poi (per l'integrazione ERP) il bearer OAuth2 client-credentials o la firma HMAC.
5. Un manager può creare/disattivare/sbloccare utenti e assegnare il ruolo (USER o MANAGER); solo MANAGER vede il Quartier Generale.

`app/core/users.py · app/core/auth.py · app/api/deps.py · app/api/endpoints/auth.py, hq.py (/users)`

4.2 — Biblioteca dei bandi (Fonte A) Fatto

1. All'avvio si carica il catalogo curato (bandi con regole compilate a mano da fonti ufficiali, con confidenza dichiarata).
2. Il cliente cerca per nome nell'elenco interno (`GET /bandi/search`, tollerante a errori di battitura).
3. Conferma (`POST /bandi/research/confirm`): registra la richiesta e controlla la cache prima di scaricare qualunque cosa.
4. Se non è in cache: scoperta senza motore di ricerca — cataloghi ufficiali (sitemap di `incentivi.gov.it` e `Invitalia`) più portali per famiglia di programma (PNRR, Erasmus+, export, energia...), poi eventuali motori a pagamento/gratuiti se configurati.
5. Download dei documenti trovati (HTML/PDF/Word), con protezioni (solo IP pubblici, redirect ricontrollati, limiti di dimensione e tempo).
6. Analisi: estrazione di requisiti collegati ai 60 criteri e delle regole numeriche; se fonti diverse danno valori diversi per la stessa regola, la regola non si pubblica e va in revisione manuale.
7. Di notte (Vercel Cron), un lavoro automatico aggiorna solo i metadati del catalogo nazionale (nessuna regola, nessun costo di analisi).

`app/core/bandi.py · discovery.py · research.py · requirements_extractor.py · catalog_job.py · app/data/bandi_catalog.py`

4.3 — Fonte B: tabelle ufficiali versionate Fatto, dati mancanti

1. Un manager carica un CSV/XLSX (CCNL, parametri, ammortamenti o benchmark) → `parse_rows` lo legge in bozza.
2. La bozza si **attesta**: va indicata la fonte ufficiale (URL o riferimento) e confermato `attest_official=True`.
3. Alla **pubblicazione**, la versione precedente con lo stesso ambito si chiude automaticamente il giorno prima della nuova.
4. A runtime, `fonte_b.scope(reference_date)` fissa — per tutta la durata di un calcolo — quale versione delle tabelle si legge: ricalcolare lo stesso budget alla stessa data produce sempre la stessa Merkle Root.
5. I criteri 1 (CCNL), 12 (maggiorazione tempo determinato), 13 (limite occasionali), 17/18 (ammortamento), 22 (benchmark prezzi), 33 (benchmark tariffa oraria) leggono da qui.

`app/core/fonte_b.py · fonte_b_admin.py · frontend: components/hq/FonteB.jsx`

Oggi in produzione non è stata caricata nessuna tabella: le voci di personale vengono respinte finché non se ne pubblica almeno una.

4.4 — Fonte C: documenti del cliente Fatto, OCR da collegare

1. Upload di un documento (busta paga, bilancio, F24): viene cifrato a riposo (AES-256-GCM, chiave `QUANTO_FILE_KEY`) prima di essere salvato.
2. Estrazione del testo: se il PDF ha un testo vero si legge direttamente (pdfplumber); se è una scansione servirebbe l'OCR (interfaccia pronta, nessun motore attivo su questo hosting).
3. Parsing per tipo di documento (funzioni dedicate per busta paga, bilancio, F24), con una percentuale di sicurezza per ogni campo letto.
4. I nomi propri, i codici fiscali e gli IBAN diventano token già in questa fase: il valore vero non viene mai salvato.
5. Sotto la soglia di confidenza (90% di default) il campo resta *da verificare* e non entra in nessun calcolo finché una persona non lo conferma o corregge.
6. Uso a valle: una busta paga confermata precompila una voce di personale nel Budget; un bilancio con le righe di costo diventa la base storica per l'Allocazione.

`app/core/fonte_c/ocr.py`, `parsers.py`, `service.py` · `app/core/crypto_store.py` · `frontend: components/Documents.jsx`

4.5 — Missione Uno: validazione del budget (60 criteri) Fatto

1. Per ogni voce di costo, si calcola l'insieme dei criteri applicabili in base a categoria e sottotipo (`applicable_criteria`).
2. Ogni criterio è una funzione indipendente in `deterministic_engine.py`: legge la riga, eventualmente una regola del bando o una tabella di Fonte B, e produce superato / non superato / **non valutato** (mai un default se manca il dato).
3. Alcuni criteri sono a livello di intero budget, non di riga (48 intensità di aiuto, 49 de minimis, 55 cash-flow, 56 variazione tra capitoli, 60 coerenza globale).
4. Il punteggio di conformità conta solo i criteri effettivamente eseguiti: un criterio non valutato non abbassa né alza il punteggio.
5. Se è configurata una chiave del modello, uno stadio aggiuntivo produce un riepilogo in prosa, ma solo dopo che ogni cifra citata è stata ricontrollata dal validatore numerico (altrimenti si usa un testo fisso).

`app/core/deterministic_engine.py` · `criteria_catalog.py` · `numeric_validator.py` · `renderer.py` · `analysis.py` · `budget_service.py`

I 60 criteri, per dominio (Modulo 11 della specifica FKOS)

Dominio	Criteri	Cosa controllano
Personale	1–15	CCNL, RAL, costo orario, TFR, oneri, superminimi, tetti orari, FTE, trasferte, straordinari, tempo determinato, occasionali, coerenza livello/attività, cross-check con la busta paga
Beni strumentali	16–30	Natura della voce, ammortamento (aliquota e pro-rata), nuovo di fabbrica, IoT 4.0/5.0, risparmio energetico, benchmark di prezzo, installazione, leasing, reverse charge, beni immateriali, manutenzione, DNSH, uso esclusivo, perizia giurata
Consulenze	31–35	Massimale sul totale, indipendenza del fornitore, benchmark tariffa oraria, codice ATECO, subappalti non autorizzati
Trasversali e di bando	36–60	Spese generali forfettarie, comunicazione, fidejussioni, revisione contabile, viaggi, penali, locazione, utenze, rappresentanza, DURC, decorrenza, non cumulabilità, intensità di aiuto/GBER, de minimis, CUP/CIG, pagamenti, IVA, cash-flow, variazione di budget, milestone, durabilità, cambio valuta, coerenza globale

4.6 — Asseverazione: Merkle e registro firmato Fatto

1. Ogni riga validata produce un hash SHA-256 canonico che combina Fonte C (il documento), Fonte B (la tabella usata), Fonte A (la regola del bando) e il calcolo stesso.
2. Gli hash delle righe formano un Albero di Merkle (con separazione tra foglie e nodi, e la foglia dispari promossa invece che duplicata): la radice è l'impronta dell'intero budget.
3. `POST /registry/register` scrive la radice in un registro a catena: ogni voce contiene l'hash della precedente ed è firmata Ed25519; un lock di database serializza le scritture per evitare corse.
4. Il registro è **append-only anche a livello di database**: un trigger Postgres impedisce fisicamente un UPDATE, un DELETE o un TRUNCATE sulla tabella, non solo la logica applicativa.
5. L'Auditor Portal ricalcola la radice in modo indipendente, la confronta con quella nel registro, verifica la firma e l'integrità dell'intera catena — anche offline, con la sola chiave pubblica.

`app/core/merkle_tree.py · registry.py · app/api/endpoints/registry.py · frontend: AuditorPortal.jsx`

Limite dichiarato: il registro è gestito da QUANTO stesso, non da un testimone terzo indipendente — la difesa è pubblicare periodicamente la chiave pubblica e l'ultimo hash fuori banda.

4.7 — Missione Due: allocazione tra i fondi Fatto

1. Si sceglie la base di calcolo: un bilancio storico caricato (Fonte C, righe con categoria e importo) oppure i fondi attivi compatibili.
2. I fondi si ricavano solo dalle regole pubblicate di un bando (categorie ammesse, intensità, finestra di ammissibilità, non cumulabilità, tetti di consulenze/spese generali) — mai da un elenco scritto a mano; dotazione massima e regime de minimis li indica un manager perché il bando spesso non li dichiara.
3. Un problema di ottimizzazione lineare intera (MILP, `scipy/HiGHS`) assegna ogni spesa a un fondo rispettando tutti i vincoli insieme, con tre possibili obiettivi e una modalità what-if.
4. Gli importi si calcolano in centesimi interi e si riverificano dopo la soluzione, per escludere errori di arrotondamento.
5. Il riepilogo testuale (se la chiave del modello è configurata) è sempre ricontrollato dal validatore numerico prima di essere mostrato.

`app/core/allocation_engine.py · funds.py · app/api/endpoints/allocation.py · frontend: AllocationView.jsx`

4.8 — Confronto: banca dei pattern e k-means Fatto, banca dati vuota

1. Un manager importa un CSV di budget storici: ogni riga deve avere la categoria del bando e la fonte (senza fonte la riga è scartata, non salvata comunque).
2. Per ogni categoria, con almeno 6 budget si calcolano gli archetipi con `k-means++` (seme fisso derivato dai dati: stesso dataset, stessi archetipi) e si sceglie il numero di gruppi con l'indice di silhouette.
3. Con meno di 6 budget si usa una sola media, dichiarata esplicitamente come tale (mai un finto clustering).
4. Un budget bozza si normalizza in quote e si confronta per similarità coseno con l'archetipo più vicino: scostamento per categoria, e i 5-10 budget storici più simili con la loro fonte.

`app/core/pattern_bank.py · app/api/endpoints/pattern.py · frontend: PatternDemo.jsx, hq/DataAdmin.jsx`

Nessuna fonte pubblica gratuita trovata finora pubblica la scomposizione per categoria di un budget reale (vedi sezione 10): la banca resta vuota finché non arriva un CSV reale.

4.9 — Quartier Generale (supervisione) Fatto

1. Ogni operazione (validazione, registrazione, ingestione, allocazione, import...) scrive un evento in `events`, ripulito da IBAN/codici fiscali prima del salvataggio.
2. Ogni validazione salva la traccia completa dell'algoritmo in `runs` (riprendibile nell'Algoritmo dall'HQ; run identiche non si duplicano).
3. Panoramica: conteggi, motore di storage, stato dell'autenticazione, timeline di ogni operazione, mappa delle operazioni.
4. Archivio bandi: ogni fonte scaricata (testo integrale + file originale), regole e requisiti modificabili a mano, scheda di controllo per un consulente, esportazione ZIP, eliminazione con ripristino dei bandi curati (tombstone).
5. Esploratore del database: lettura ed export CSV solo per le tabelle in whitelist; eliminazione riga tramite un identificativo codificato (Postgres non ha un rowid implicito); il registro `anchors` è protetto anche qui.

`app/core/hq.py` · `archive.py` · `events.py` · `frontend: components/hq/*`

4.10 — Intelligenza artificiale (opzionale) Pronto, spento senza chiave

1. Stadio 1 (arricchimento): quando un bando viene scoperto, un passaggio opzionale ne legge i metadati principali.
2. Stadio 3 (letture multiple): N passaggi indipendenti (di solito 3) leggono la stessa regola del bando; ognuno deve riportare la frase esatta del documento sorgente o il valore viene scartato.
3. Il confronto tra i passaggi lo fa sempre il codice, non l'IA: se sono d'accordo la regola si pubblica, altrimenti va in coda di revisione umana.
4. Riepiloghi in prosa (Budget e Allocazione): generati solo dopo il calcolo, e ogni cifra citata è ricontrollata dal validatore numerico — se non torna, si usa un testo fisso invece del testo generato.

`app/core/llm.py` · `numeric_validator.py` · `renderer.py`

Inerte per intero senza `ANTHROPIC_API_KEY`: nessuna chiamata parte, tutto resta deterministico.

4.11 — Lavoro notturno (Cron) Fatto

1. Vercel Cron chiama `GET /api/v2/cron/catalog-refresh` ogni notte alle 3:00, protetto da `CRON_SECRET`.
2. Aggiorna solo i metadati del catalogo nazionale dei bandi (nessuna regola, nessuna estrazione costosa).
3. Se è configurata una chiave del modello, arricchisce in più le nuove voci con Stadio 1.

`app/core/catalog_job.py` · `app/api/endpoints/cron.py` · `vercel.json (crons)`

4.12 — Esportazione e integrazione Fatto

1. Export XLSX e PDF del budget certificato, con identificativo (CEP-ID) e QR code per la verifica.
2. Webhook firmati HMAC verso sistemi esterni.
3. Autenticazione ERP separata da quella utente: OAuth2 client-credentials (JWT) o firma HMAC delle richieste.

`app/core/export.py` · `app/core/webhooks.py` · `app/api/endpoints/auth.py`

5. Come sono state costruite le parti già fatte

Non solo cosa fanno, ma *perché* sono fatte così — utile per chi deve estenderle senza romperne le garanzie.

Il livello database

Postgres ha sostituito SQLite senza riscrivere le centinaia di punti che chiamavano `conn.execute(sql, params).fetchone()` [`'col'`]: `app/core/db.py` definisce una classe `Row` e una `Conn` che imitano esattamente quell'interfaccia, più un traduttore che trasforma i segnaposto `?` di SQLite in `%s` di Postgres al volo. Le migrazioni (`schema.py`) sono numerate e girano sotto un lock consultivo di Postgres, così più istanze che partono insieme non si pestano i piedi. Postgres non ha un `rowid` implicito come SQLite: dove serviva (l'esploratore del database dell'HQ) si è codificata l'identità di riga come base64 di un JSON delle colonne chiave.

Il registro non doveva fidarsi solo del codice applicativo

Impedire un UPDATE o un DELETE sulla tabella `anchors` solo nella logica Python avrebbe lasciato un varco a chiunque avesse accesso diretto al database. Per questo l'append-only è imposto anche da un **trigger Postgres** (`BEFORE UPDATE OR DELETE` e `BEFORE TRUNCATE`) che solleva un errore: è una garanzia del database, non una convenzione del codice.

Fonte B usa un contesto, non un parametro ovunque

Passare `reference_date` a mano in ogni funzione che poteva leggere una tabella CCNL sarebbe stato fragile (basta dimenticarlo una volta). Si è scelto un `contextvars.ContextVar`: `fonte_b.scope(reference_date)` apre un contesto per tutta la durata di un calcolo, e ogni lettura di tabella dentro quel contesto vede la stessa versione — anche se nel frattempo un manager ne pubblica una nuova. È la garanzia che ricalcolare lo stesso budget alla stessa data produca sempre la stessa Merkle Root.

Fonte C: cifrare a riposo e tokenizzare in lettura, non "poi"

La chiave di cifratura dei file (`QUANTO_FILE_KEY`) è separata da quella di firma del registro e da quella dei token PII: se una viene compromessa, le altre due garanzie restano in piedi. La tokenizzazione di nomi, codici fiscali e IBAN avviene durante la lettura del documento, non dopo: il valore vero non viene mai scritto su disco, nemmeno per un istante in più del necessario per calcolare il token.

Il motore a 60 criteri: funzioni pure, "non valutato" come terzo stato

Ogni criterio in `deterministic_engine.py` è una funzione indipendente che riceve la riga (e, se serve, la regola del bando o la tabella di Fonte B) e produce uno tra tre esiti: superato, non superato, **non valutato**. La distinzione tra "non superato" e "non valutato" è la scelta più importante del motore: un criterio senza il dato necessario non viene mai contato come superato per far salire il punteggio, né come fallito per un dato che semplicemente non c'è ancora. Aggiungere un criterio significa: una funzione nuova qui, una voce nel dizionario `CRITERIA_TITLES` (`criteria_catalog.py`), e — se dipende da una regola del bando — una voce in `RULE_CRITERIA` (`bandi.py`).

Autenticazione: script e revoca per versione, non lista nera

`hashlib.scrypt` della libreria standard di Python evita una dipendenza esterna per l'hashing delle password. Il blocco dopo 5 tentativi è nel database (non in memoria del processo), quindi funziona anche con più istanze del server attive insieme. La revoca dei token non usa una lista nera che cresce all'infinito: ogni token porta il `token_version` dell'utente al momento dell'emissione, e cambiare password o disattivare l'utente incrementa quel numero — ogni token precedente smette di funzionare all'istante, con un solo confronto.

Pattern bank: k-means riproducibile, non casuale

Il seme del clustering non è casuale: è derivato con SHA-256 dalla categoria e dal numero di budget presenti. Lo stesso insieme di dati produce sempre gli stessi archetipi, in qualunque momento si ricalcoli — una proprietà necessaria perché il confronto di un budget con l'archetipo sia riproducibile e verificabile, non dipendente da quando è girato il calcolo.

LLM: la citazione verbatim come garanzia strutturale, non come promessa

Ogni valore che il modello estrae da un documento deve arrivare insieme alla frase esatta del testo da cui proviene; il codice cerca quella frase (ripulita da spazi/accenti) nel documento sorgente, e se non la trova **scarta il valore**, a prescindere da quanto sembri plausibile. È una barriera strutturale contro le allucinazioni, verificata dal codice e non affidata al modello stesso.

Ricerca dei bandi: perché "senza motore di ricerca" come prima via

I motori di ricerca gratuiti spesso bloccano le richieste automatiche quando arrivano da un IP di un provider cloud come Vercel (si è osservato Bing rispondere con risultati senza alcun legame con la query). Per questo la scoperta di un bando parte dai cataloghi ufficiali (sitemap di siti istituzionali) e dai portali per famiglia di programma, seguendo i link che nominano il bando per due salti — e usa un motore di ricerca vero e proprio solo come rinforzo, se è stata configurata una chiave di un servizio a pagamento/free-tier (Brave, Serper, Tavily).

6. Come tutto è collegato

Ogni endpoint è un file sottile: valida l'input con Pydantic, chiama uno o due moduli di `app/core`, restituisce la risposta. La tabella seguente è la mappa per orientarsi rapidamente da un endpoint al codice che conta davvero.

Endpoint (prefisso <code>/api/v2</code>)	Modulo core principale	Tabelle toccate
<code>/auth/login</code> , <code>/auth/me</code> , <code>/auth/change-password</code>	<code>users.py</code> , <code>auth.py</code>	<code>users</code>
<code>/bandi/*</code>	<code>bandi.py</code> , <code>discovery.py</code> , <code>research.py</code>	<code>bandi</code> , <code>bando_meta</code> , <code>rules</code> , <code>requirements</code> , <code>bando_sources</code> , <code>bando_files</code> , <code>bando_tombstones</code>
<code>/fonte-b/*</code>	<code>fonte_b.py</code> , <code>fonte_b_admin.py</code>	<code>fonte_b_datasets</code> (CCNL/parametri/ammortamenti/benchmark, versionate)
<code>/fonte-c/*</code>	<code>fonte_c/service.py</code> , <code>parsers.py</code> , <code>ocr.py</code>	<code>documents</code> (contenuto cifrato) + tabella dei campi estratti
<code>/budget/validate</code>	<code>deterministic_engine.py</code> , <code>analysis.py</code>	<code>runs</code> , <code>events</code> (lettura da <code>fonte_b_datasets</code> in scope)
<code>/registry/register</code> , <code>/registry/verify</code> , <code>/registry/status</code>	<code>merkle_tree.py</code> , <code>registry.py</code>	<code>anchors</code> (append-only)
<code>/allocation/optimize</code> , <code>/allocation/funds*</code>	<code>allocation_engine.py</code> , <code>funds.py</code>	<code>funds</code> , <code>runs</code>
<code>/pattern/match</code> , <code>/pattern/budgets</code> , <code>/pattern/import</code>	<code>pattern_bank.py</code>	<code>pattern_budgets</code> , <code>pattern_archetypes</code>
<code>/hq/*</code>	<code>hq.py</code> , <code>archive.py</code> , <code>events.py</code>	<code>events</code> , <code>runs</code> , <code>documents</code> , <code>bando_*</code> (lettura/gestione)
<code>/cron/catalog-refresh</code>	<code>catalog_job.py</code>	<code>bandi</code> (solo metadati CAT-*)

Collanti trasversali

- `app/api/deps.py` — usato da quasi ogni endpoint protetto: `require_auth` (token utente → bearer ERP → firma HMAC, in quest'ordine), `actor_of` (chi ha fatto l'operazione, per il log eventi), `require_hq` (in più, richiede il ruolo MANAGER).
- `app/core/events.py` — bus di audit: quasi ogni modulo core chiama `events.record(...)` alla fine di un'operazione; l'HQ legge da qui la timeline e la mappa delle operazioni.
- `fonte_b.scope(...)` — contesto condiviso (vedi sezione 5) tra `analysis.py` e `deterministic_engine.py` senza passare la data ovunque esplicitamente.
- `frontend/src/lib/api.js` — unico punto di contatto col backend: un wrapper per ogni endpoint, il token della sessione allegato automaticamente, e un evento globale `quanto-logout` emesso a ogni 401 (sessione scaduta o revocata) che `App.jsx` intercetta per riportare l'utente al login.
- `frontend/src/lib/nav.js` — un `NavContext` minimo che permette a un componente profondo (es. un pulsante dentro Documenti) di cambiare scheda e passare un'ancora alla Guida, senza passare callback attraverso ogni livello.

7. Come è composta la pagina (frontend)

`App.jsx` è il guscio unico dell'applicazione: non c'è un router — la "pagina" attiva è semplicemente una stringa in uno `useState (tab)`, sincronizzata con il parametro `?tab=` nell'URL. Tutto lo stato che serve a più schede insieme (il bando scelto, la richiesta di budget corrente, l'ultima validazione, l'utente loggato) vive qui e scende ai componenti come prop — nessuna libreria di gestione dello stato: la app è abbastanza piccola da non averne bisogno.

Struttura visiva della pagina

Header — sticky in alto: logo (torna a "Bandi"), barra di schede orizzontale (icona Lucide + testo, scorrevole su mobile), a destra nome utente, cambio password, uscita.

Intestazione di pagina (PageHead) — icona, titolo e sottotitolo in una riga sola ("a cosa serve questa pagina, in parole semplici"), più due indicatori sempre visibili: lo stato del registro (integro / alterato / chiave di prova) e il bando attualmente in uso.

Contenuto — un solo componente alla volta, quello della scheda attiva (vedi tabella sotto).

Livello sovrapposto — le finestre modali (cambio password, registrazione della Merkle Root) vivono fuori dal flusso normale, condizionate da un booleano di stato.

Le schede e i loro componenti

Scheda	Componente principale	Componenti figli / usati
Bandi	<code>BandiLibrary.jsx</code>	ricerca interna + conferma, lettura dettaglio (fonti, regole, requisiti, copertura dei 60 criteri)
Documenti	<code>Documents.jsx</code>	upload, elenco documenti con stato di confidenza per campo, revisione (conferma/correggi)
Budget	<code>BudgetCanvas.jsx</code>	<code>LineEditor.jsx</code> (editor di riga guidato dal catalogo campi), <code>IngestionPanel.jsx</code> (import Excel), pulsante verso l'Algoritmo
Algoritmo (non in barra)	<code>AlgorithmLab.jsx</code>	raggiunta solo dal pulsante "Guarda come ha lavorato": fasi, mappa criteri×voci, cascata importi, albero di Merkle
Allocazione	<code>AllocationView.jsx</code>	selezione bilancio storico/fondi, risultato MILP, what-if
Confronto	<code>PatternDemo.jsx</code>	selettore categoria, confronto live (debounce), lista budget storici più simili
Verifica	<code>AuditorPortal.jsx</code>	pubblica, nessun login: ricalcolo radice e confronto col registro
Guida	<code>Guida.jsx</code> , <code>Guide.jsx</code> , <code>Help.jsx</code>	testi da <code>data/help.js</code> , con ancore che le altre pagine possono richiamare
Quartier Generale	<code>hq/HQ.jsx</code>	<code>hq/HQSections.jsx</code> (panoramica, timeline), <code>hq/Users.jsx</code> , <code>hq/FonteB.jsx</code> , <code>hq/DataAdmin.jsx</code> (pattern/fondi/catalogo), <code>hq/Archive.jsx</code>

Sistema grafico

Bianco come colore principale, uno stile "vetro liquido" ispirato al linguaggio visivo Apple più recente (classi `.glass`, `.glass-strong`, `.card`) e i due gialli del marchio (`#f1e21b`, `#ffffb96`) usati in gradiente per gli elementi attivi (`.bg-liquid`). Icone Lucide a tratto sottile in tutta l'app. I token di colore/spaziatura sono in `tailwind.config.js` e `src/index.css`; il font di marchio "Now" non essendo su Google Fonts richiede i file in `frontend/public/fonts` (altrimenti si usa Outfit come alternativa). Componenti condivisi tra pagine: `ui.jsx` (logo, intestazioni, elementi di base), `data/ruleLabels.js` (nomi leggibili delle regole), `lib/format.js` (formattazione di valuta/numero, mai un calcolo).

Ogni pagina ha, nei suoi testi, una spiegazione in parole semplici e un esempio (`data/help.js`): è il posto giusto dove aggiungere la spiegazione di una nuova funzione quando se ne costruisce una.

8. Stato attuale, in una tabella

Area	Stato	Nota
Missione Uno — 60 criteri	Fatto	Un criterio senza dato è "non valutato", mai "superato"; punteggio calcolato solo sui criteri eseguiti
Asseverazione (Merkle + registro)	Fatto	Append-only anche via trigger di database; verifica offline con sola chiave pubblica
Missione Due (allocazione)	Fatto	Fondi derivati dalle regole pubblicate dei bandi, mai da un elenco scritto a mano
Autenticazione	Fatto	Email e password, script, blocco e revoca; attiva di serie in produzione
Persistenza	Fatto	PostgreSQL esterno (Supabase); nessun dato si perde ai riavvii
Fonte A — ricerca bandi	Fatto	9 bandi curati in produzione; ricerca interna → conferma → cache → web
Fonte B — tabelle ufficiali	Parziale	Meccanismo pronto e provato dai test; nessuna tabella caricata in produzione
Fonte C — documenti cliente	Parziale	Lettura di PDF con testo vero pronta; OCR per le scansioni da collegare (Tesseract non gira su Vercel)
Banca dei pattern	Parziale	k-means pronto e provato; banca dati vuota — nessuna fonte pubblica gratuita adeguata trovata finora
Intelligenza artificiale	Parziale	Modulo pronto (Stadio 1, Stadio 3, riepiloghi); spento senza <code>ANTHROPIC_API_KEY</code>
Forma dell'API	Parziale	Allocazione e Confronto allineati ai nomi di campo dello schema originale; <code>/budget/validate</code> non ancora toccato
Console, report di asseverazione in PDF, WP, audit di calibrazione, ricerca semantica	Non iniziato	Priorità bassa/futura, non richiesti finora

9. Convenzioni per chi ci lavora

- **Mai un dato inventato.** Se manca la fonte, il criterio resta "non valutato", la regola resta "da verificare", il fondo non si crea. Questo vale per ogni nuova funzione: prima la fonte reale, poi il codice che la usa.
- **Denaro sempre `Decimal`**, mai `float` — in tutto il backend, senza eccezioni.
- **Ogni regola ha una fonte e una confidenza** (PRIMARIA = testo ufficiale letto direttamente, SECONDARIA = fonte non ufficiale da verificare, INTERPRETAZIONE = traduzione in regola eseguibile di un testo qualitativo) — si veda `app/data/bandi_catalog.py` per lo schema esatto di una scheda bando.
- **Testi in italiano semplice**, non burocratese, sia nei commenti del codice sia nell'interfaccia: è lo stile già adottato ovunque, da mantenere per coerenza.
- **Test su Postgres vero**, non SQLite in memoria: in locale `pixeltable-pgserver` incorpora un Postgres vero per i test, in CI gira un servizio `postgres:16`. Si lancia con `pytest` dalla cartella `backend` (ambiente virtuale in `backend/.venv`).
- **Aggiungere un criterio:** una funzione in `deterministic_engine.py`, una voce in `CRITERIA_TITLES` (`criteria_catalog.py`), e se dipende da una regola del bando una voce in `RULE_CRITERIA` (`bandi.py`).
- **Aggiungere un bando curato:** una scheda in `app/data/bandi_catalog.py` (funzione `R()` per ogni requisito), con fonte e confidenza per ogni regola, e i punti mancanti dichiarati in `not_specified` — non lasciati impliciti.
- **Deploy:** `git push origin main`, poi `vercel --prod` dalla root del repository (o si lascia al collegamento automatico GitHub → Vercel); le variabili d'ambiente obbligatorie e facoltative sono elencate per intero in `docs/DEPLOY.md`.

10. Da dove continuare

In ordine di impatto per sbloccare funzionalità già pronte nel codice ma inerti in produzione:

1. **Caricare almeno una tabella Fonte B ufficiale** (Quartier Generale → Tabelle ufficiali). Finché non c'è, ogni voce di personale nel Budget viene respinta con "tabella CCNL non disponibile".
2. **Procurare un CSV storico reale per la banca dei pattern.** Verificato di persona: OpenCoesione pubblica solo importi totali (nessuna scomposizione personale/beni/consulenze/formazione/comunicazione); un bilancio aziendale ha lo stesso limite. Serve un CSV da un pilota reale, un commercialista o un'associazione di categoria — inventare la scomposizione da un totale violerebbe il principio "niente dati finti".
3. **Decidere un servizio OCR in cloud** per le scansioni di Fonte C (Tesseract non gira su Vercel).
4. **Decidere se attivare** `ANTHROPIC_API_KEY` per lo Stadio 1/3 e i riepiloghi in prosa.
5. **Allineare** `/budget/validate` ai nomi di campo dello schema originale, se si vuole coerenza completa con la specifica (oggi solo Allocazione e Confronto sono stati allineati, in modo additivo).
6. Voci a priorità più bassa, non iniziate: numeri cliccabili nella Console, export dell'allocazione, report di asseverazione in PDF scaricabile, pacchetti di lavoro, audit di calibrazione, ricerca semantica, separazione in container.

Per orientarsi in fretta nel codice: partire da `app/api/router.py` per l'elenco degli endpoint, da `app/core/analysis.py` per il percorso di una validazione completa, e da `frontend/src/App.jsx` per la struttura della pagina. Il `README.md` del repository resta la referenza rapida per i comandi e lo stato di implementazione modulo per modulo.